PROJECT DOCUMENTATION

Deep Learning for Comment Toxicity Detection with Streamlit

Domain: Online Community Management & Content Moderation

Tech Stack: Python | TensorFlow | Bidirectional LSTM | Streamlit | NLP

Author: Manikandan | GitHub: Manikandan-08J | 2025

1. Project Overview

Field	Details
Project Title	Deep Learning for Comment Toxicity Detection with Streamlit
Domain	Online Community Management & Content Moderation
Skills Covered	Deep Learning, NLP, Model Training, Streamlit Deployment
Tech Stack	Python, TensorFlow, Keras, Streamlit, Pandas, NumPy, Matplotlib
Model Type	Bidirectional LSTM (Multi-label Classification)
Output	Real-time Interactive Toxicity Detection Web Application
Timeline	One Week

2. Problem Statement

Online communities and social media platforms have become integral parts of modern communication, facilitating interactions and discussions on various topics. However, the prevalence of toxic comments — including harassment, hate speech, and offensive language — poses significant challenges to maintaining healthy and constructive online discourse.

This project develops a deep learning-based comment toxicity model using Python. The model analyzes text input from online comments and predicts the likelihood of each comment being toxic across 6 distinct categories. By accurately identifying toxic comments, the model assists platform moderators and administrators in taking appropriate actions to mitigate the negative impact of toxic behavior.

3. Business Use Cases

3.1 Target Industries

- Social Media Platforms — Auto-detect and filter toxic comments in real-time
- Online Forums & Communities — Moderate user-generated content efficiently at scale
- Content Moderation Services — Enhance moderation capabilities for third-party platforms
- Brand Safety & Reputation Management — Ensure ads appear in safe online environments
- E-learning Platforms — Create safer online learning environments for students
- News Websites & Media Outlets — Moderate user comments on articles and posts

4. Project Structure

```
Comment_Toxicity_Detection_with_Streamlit/
    data/
        train.csv                   # Training dataset (Jigsaw Kaggle)
    models/
        toxicity_model.h5            # Trained Bidirectional LSTM model
        tokenizer.pkl                # Fitted Keras tokenizer
        metrics.json                 # AUC-ROC & training history
    utils/
        __init__.py
        preprocess.py               # Text cleaning functions
        predict.py                  # Single & batch prediction logic
    c_t_app.py                      # Streamlit web application
    toxicity_project.ipynb          # Model training notebook
    eda_report.png                  # EDA visualization chart
    requirements.txt                # Python dependencies
    README.md                       # Project documentation
```

5. Dataset

5.1 Source

The dataset used is the Jigsaw Toxic Comment Classification dataset from Kaggle. It contains Wikipedia talk page comments labeled by human raters for toxic behavior.

5.2 Dataset Structure

Column	Type	Description
id	String	Unique identifier for each comment
comment_text	String	Raw text of the online comment
toxic	Integer (0/1)	General toxic content label
severe_toxic	Integer (0/1)	Severely toxic content label
obscene	Integer (0/1)	Obscene language label
threat	Integer (0/1)	Threatening language label
insult	Integer (0/1)	Insulting content label
identity_hate	Integer (0/1)	Hate speech targeting identity

5.3 EDA Findings

- Dataset is heavily imbalanced — approximately 90% clean comments, 10% toxic
- toxic is the most common label (~9.6%), identity_hate is the rarest (~0.3%)

- Comment lengths vary widely — most comments are under 500 characters
- Multiple labels can apply to a single comment simultaneously (multi-label)

6. Technical Approach

6.1 Data Preprocessing (utils/preprocess.py)

The clean_text() function applies the following transformations:

- Convert all text to lowercase
- Remove URLs (http/www patterns)
- Strip HTML tags using regex
- Remove newline, carriage return, and tab characters
- Keep only alphanumeric characters and basic punctuation (!?.,')
- Collapse multiple spaces into single spaces and strip whitespace

6.2 Tokenization & Sequence Preparation

Parameter	Value	Reason
MAX_VOCAB	50,000	Covers most common English words
MAX_LEN	200	Covers ~95% of comment lengths
OOV Token	<OOV>	Handles unseen words at inference
Train/Val Split	90% / 10%	Manual random split with seed=42
Batch Size	256	Efficient GPU utilization

6.3 Model Architecture

A Bidirectional LSTM model was chosen for its ability to capture contextual information from both directions in a text sequence, which is crucial for understanding toxic intent in comments.

Layer	Configuration	Output Shape
Input	Integer sequences	(None, 200)
Embedding	50001 vocab, 128 dims, trainable=True	(None, 200, 128)
Bidirectional LSTM	64 units, return_sequences=True	(None, 200, 128)
GlobalMaxPooling1D	-	(None, 128)
Dense	64 units, ReLU activation	(None, 64)
Dropout	rate=0.3	(None, 64)
Output Dense	6 units, Sigmoid activation	(None, 6)

- Loss Function: Binary Crossentropy (multi-label classification)
- Optimizer: Adam with default learning rate
- Metrics: Accuracy per epoch
- Threshold: 0.5 probability for positive classification

6.4 Training Configuration

- Epochs: Up to 5 (with EarlyStopping patience=2)
- EarlyStopping: Restores best weights automatically
- ModelCheckpoint: Saves best model to models/toxicity_model.h5
- Validation: 10% hold-out set monitored every epoch

6.5 Prediction Logic (utils/predict.py)

Single Comment Prediction

- Clean text using preprocess.py
- Tokenize and pad to MAX_LEN=200
- Run model.predict() to get 6 probability scores
- Apply threshold=0.5 to determine is_toxic flag
- Identify dominant_label as the category with highest probability

Batch CSV Prediction

- Apply same pipeline to all rows in the uploaded CSV
- Return a DataFrame with toxicity scores for all 6 categories
- Add is_toxic and dominant_label columns to output

7. Streamlit Application (c_t_app.py)

7.1 Features

- Single Comment Analysis — Text input box for real-time toxicity predictions
- Toxicity Score Display — Shows probability for all 6 toxicity categories
- Dominant Label Detection — Highlights the most prominent toxicity type
- Bulk CSV Upload — Upload a CSV file for batch predictions
- EDA Dashboard — Visualizes dataset label distribution and model metrics
- Interactive Charts — Plotly visualizations for scores and trends

7.2 How to Run Locally

```
# Step 1: Navigate to project folder
cd Comment_Toxicity_Detection_with_Streamlit
```

```
# Step 2: Activate virtual environment (Windows)
.venv\Scripts\activate
```

```
# Step 3: Run the Streamlit app
streamlit run c_t_app.py
```

The application opens at: http://localhost:8501

8. Model Evaluation

8.1 Metrics Used

- AUC-ROC per label — measures discriminatory ability for each toxicity category
- Classification Report — Precision, Recall, F1-score per label
- Training & Validation Loss curves — to monitor for overfitting
- Training & Validation Accuracy curves — epoch-by-epoch performance

8.2 Evaluation Code

```
y_pred     = model.predict(X_val_seq, batch_size=256)
y_pred_bin = (y_pred > 0.5).astype(int)
auc = roc_auc_score(y_val[:, i], y_pred[:, i])
# Results saved to models/metrics.json
```

8.3 Sample Test Results

Comment	Verdict	Dominant Label
You are so stupid and worthless!	TOXIC	toxic (96.4%)
I will find you and hurt you.	CLEAN	-
Thank you for the helpful explanation.	CLEAN	-
Great work on this project!	CLEAN	-

9. Deployment Guide

9.1 Local Setup

```
git clone
https://github.com/Manikandan-08J/Deep_Learning_for_Comment_Toxicity_Detection_
with_Streamlit.git
cd Deep_Learning_for_Comment_Toxicity_Detection_with_Streamlit
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
streamlit run c_t_app.py
```

9.2 Streamlit Cloud Deployment

- Push all code to GitHub (public repository)
- Go to share.streamlit.io and sign in with GitHub
- Click New App and select your repository
- Set the main file path to c_t_app.py
- Click Deploy — Streamlit Cloud reads requirements.txt automatically

9.3 Requirements

Package	Purpose
tensorflow	Deep learning model (BiLSTM)
streamlit	Web application framework
pandas	Data manipulation & CSV handling
numpy	Numerical computations
scikit-learn	AUC-ROC and classification metrics
plotly	Interactive charts in Streamlit
matplotlib	EDA visualization charts

10. Coding Standards & Best Practices

- Code follows PEP 8 Python style guidelines
- Modular structure — logic split into utils/preprocess.py and utils/predict.py
- Functions are documented with clear input/output contracts
- Constants (MAX_LEN, MAX_VOCAB, THRESHOLD, LABEL_COLS) defined at module level
- All model artifacts saved to models/ folder for reproducibility
- Random seed (42) set for reproducible train/val splits
- EarlyStopping and ModelCheckpoint used for robust training

11. GitHub Repository

Item	Details
Repository	Deep_Learning_for_Comment_Toxicity_Detection_with_Streamlit
GitHub User	Manikandan-08J
Visibility	Public
Branch	main
Main App File	c_t_app.py
Notebook	toxicity_project.ipynb

11.1 Push to GitHub Commands

```
git add .
git commit -m "Add project files"
git push origin main
```

12. Project Deliverables

Deliverable	Status	Description
Streamlit Application	Completed	Interactive web app for real-time toxicity predictions
Trained Model (.h5)	Completed	Bidirectional LSTM saved to models/toxicity_model.h5
Tokenizer (.pkl)	Completed	Fitted Keras tokenizer saved to models/tokenizer.pkl
Preprocessing Module	Completed	utils/preprocess.py with clean_text() function
Prediction Module	Completed	utils/predict.py with single & batch prediction
EDA Report	Completed	eda_report.png with 3-panel visualization
Metrics JSON	Completed	models/metrics.json with AUC and training history
README	Completed	Comprehensive README.md on GitHub
Documentation	Completed	This project documentation file
LinkedIn Demo Video	Pending	Demo video to be posted on LinkedIn

13. References

- Jigsaw Toxic Comment Classification Challenge — Kaggle
- TensorFlow / Keras Documentation — tensorflow.org

- Streamlit Documentation — docs.streamlit.io
- PEP 8 Python Style Guide — python.org/dev/peps/pep-0008
- Bidirectional LSTM Paper — Schuster & Paliwal, 1997